

Chain of Thought (CoT)

Módulo 6 · Ingeniería de Prompts

Zero-shot CoT, few-shot CoT, CoT con XML, auto-refinamiento y cuándo el razonamiento paso a paso mejora realmente el rendimiento.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Chain of Thought (CoT)

En 2022, Wei et al. publicaron "Chain of Thought Prompting Elicits Reasoning in Large Language Models", demostrando que simplemente pedir al modelo que razonara paso a paso antes de responder mejoraba dramáticamente el rendimiento en tareas de razonamiento aritmético y lógico. Este hallazgo, aparentemente simple, cambió radicalmente cómo los profesionales usan los LLMs en producción. Chain of Thought es hoy una de las técnicas más documentadas y aplicadas en prompt engineering.

La intuición detrás de CoT es sólida: el razonamiento multi-paso es difícil de comprimir en una sola predicción de token. Al externalizar los pasos intermedios como texto generado, el modelo puede "prepararse" para cada siguiente paso con la información de los pasos anteriores en el contexto. Es la diferencia entre pedir a alguien que multiplique 237×48 mentalmente en un segundo y darle papel y lápiz para calcular.

Las variantes de Chain of Thought

Zero-shot CoT: el simple "piensa paso a paso"

Kojima et al. (2022) demostraron que añadir simplemente "Vamos a razonar paso a paso" (Let's think step by step) al final de un prompt mejoraba significativamente el rendimiento en problemas de razonamiento en modelos grandes. Esta variante zero-shot es la más fácil de implementar: no requiere ejemplos y funciona "out of the box" con la mayoría de los LLMs modernos. Es el primer paso que debe probarse antes de técnicas más complejas.

Few-shot CoT: demostrar el razonamiento con ejemplos

En few-shot CoT, se proporcionan 3-8 ejemplos donde cada ejemplo incluye no solo la respuesta final sino toda la cadena de razonamiento intermedio que lleva a ella. Esto enseña al modelo exactamente qué tipo de razonamiento se espera: qué pasos dar, qué nivel de detalle, cómo verificar el trabajo. Few-shot CoT produce mejores resultados que zero-shot CoT en tareas con estructura de razonamiento muy específica.

CoT con etiquetas XML: separar razonamiento de respuesta

Una práctica efectiva es instruir al modelo a generar su razonamiento dentro de etiquetas específicas: "Razona dentro de ... y luego da tu respuesta final dentro de ...". Esto separa claramente el proceso de razonamiento (que puede ser largo) de la respuesta final (que debe ser concisa). Permite extraer solo la respuesta para el usuario final mientras el razonamiento se guarda para auditoría.

Por qué funciona el CoT y cuándo aplicarlo

CoT funciona bien para tareas que requieren múltiples pasos de razonamiento donde cada paso depende de los anteriores: problemas matemáticos multi-paso, razonamiento lógico complejo,

análisis que requieren considerar múltiples factores, diagnósticos que implican descartar hipótesis. No añade valor significativo para tareas simples de una sola operación o para recuperación directa de información del contexto.

La investigación ha establecido dos condiciones para que CoT sea efectivo: el modelo debe ser suficientemente grande (los beneficios son más pronunciados en modelos de 7B+ parámetros) y la tarea debe genuinamente requerir razonamiento multi-paso. Para tareas simples de clasificación o extracción, el overhead de CoT puede incluso perjudicar el rendimiento al consumir tokens de respuesta innecesariamente.

Auto-refinamiento y CoT iterativo

Una extensión natural de CoT es el auto-refinamiento: después de generar una respuesta con razonamiento, se pide al mismo modelo que critique y mejore su propia respuesta. "Revisa tu respuesta anterior. ¿Hay algún error lógico? ¿Hay factores que no has considerado? Si es así, proporciona una respuesta mejorada." Este ciclo puede ejecutarse 1-3 veces para mejorar la calidad en tareas de alta importancia.

SECCIÓN 4 · EJEMPLOS REALES

- Análisis de riesgo contractual con CoT: "Analiza el siguiente contrato para identificar cláusulas de riesgo. Razona paso a paso: 1) identifica las partes y sus obligaciones, 2) analiza las cláusulas de penalización, 3) evalúa las condiciones de rescisión, 4) concluye sobre el nivel de riesgo general." El razonamiento estructurado produce análisis más completos que una pregunta directa.
- Diagnóstico de bugs con CoT: GitHub Copilot Chat usa implícitamente CoT cuando se le pide ayuda con errores complejos: "Analiza el stack trace, identifica la causa raíz probable, explica por qué ocurre el error, y sugiere las correcciones más probables." El razonamiento multi-paso produce diagnósticos más precisos que una respuesta directa.
- Estimación de proyectos de software: "Descompón este proyecto en tareas. Para cada tarea, estima el tiempo en días considerando la complejidad técnica y las dependencias. Suma las estimaciones individuales y añade un margen del 20% para imprevistos." El CoT fuerza una descomposición sistemática que reduce la subestimación.
- Evaluación de candidatos en RRHH: con CoT, el modelo evalúa cada criterio de selección de forma sistemática antes de producir una puntuación final, produciendo evaluaciones más objetivas y auditables que una valoración holística directa.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Aplicar CoT a tareas simples. Para una pregunta de clasificación binaria o una extracción directa de un dato del contexto, añadir "piensa paso a paso" añade tokens sin mejorar el resultado. El overhead de CoT debe justificarse con la complejidad genuina de la tarea.

Error 2: Confundir CoT con razonamiento correcto garantizado. CoT mejora el razonamiento estadísticamente pero no garantiza que el razonamiento sea correcto. El modelo puede construir una cadena de razonamiento plausible pero incorrecta con la misma fluidez que una correcta. La verificación del razonamiento (auto-refinamiento,

self-consistency, evaluación humana) sigue siendo necesaria.

Error 3: Asumir que el razonamiento visible refleja el razonamiento real del modelo. El texto de razonamiento generado por CoT es output del modelo, no una ventana al proceso computacional real. Puede ser útil para auditoría pero no revela los mecanismos internos del modelo.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Guía de implementación: para zero-shot CoT, simplemente añade al final del prompt "Razona paso a paso antes de responder" o "Piensa en voz alta antes de dar tu respuesta final". Para few-shot CoT, construye 3-5 ejemplos que demuestren el tipo de razonamiento esperado, incluyendo los pasos intermedios. Para CoT con estructura XML, usa etiquetas como

Para evaluar si CoT añade valor en tu caso de uso: crea un golden dataset de 30-50 casos con respuestas correctas conocidas. Ejecuta el mismo prompt con y sin CoT. Si la precisión mejora más del 3-5%, CoT vale el coste adicional de tokens. Si la mejora es marginal, evalúa si la mejora en auditabilidad (razonamiento visible) justifica el coste.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M6-T3 (ToT): Tree of Thought es una extensión de CoT que explora múltiples cadenas de razonamiento en paralelo.
- M6-T4 (Self-consistency): combina múltiples instancias de CoT y selecciona por mayoría.
- M1-T4 (Sistema 1 vs 2): CoT es la forma de inducir comportamiento Sistema 2 (deliberativo) en modelos que por defecto operan en modo Sistema 1 (rápido).

SECCIÓN 8 · RESUMEN EJECUTIVO

Chain of Thought prompting mejora el rendimiento en tareas de razonamiento multi-paso al externalizar los pasos intermedios como texto generado. Zero-shot CoT: simplemente añade "piensa paso a paso". Few-shot CoT: proporciona ejemplos con el razonamiento completo. CoT con XML: separa razonamiento de respuesta para auditoría. Funciona mejor en modelos grandes y tareas genuinamente complejas. No añade valor en tareas simples. No garantiza razonamiento correcto: complementar con self-consistency o auto-refinamiento para casos críticos. Valor adicional: auditoría del razonamiento para sistemas regulados.